Futrix AI — Technical PRD

Oct 12, 2026 · @nitesh

1. Product summary

Futrix AI is an autonomous career intelligence platform: a user uploads a resume and receives their skills, skill gaps, a 0–100 readiness score, an ATS audit and matched career paths in one analysis. The analysis is deterministic, so the same resume always gives the same result and no skill is ever invented.

One-line pitch: Know exactly why your resume gets filtered, what skills you lack for your target role, and what to learn next.

Problem

- Job seekers send resumes without knowing whether an applicant tracking system (ATS) will parse them correctly.
- Students and early-career engineers cannot tell which skills separate them from a target role.
- Existing advice is generic, and tools built on generative models can give different or invented answers on each run.

Solution

Need	Futrix AI capability
"Which skills do I have?"	Skill extraction from a curated database of 40+ technologies
"What am I missing?"	Gap detection against role benchmarks, plus a learning roadmap
"Am I ready?"	Composite readiness score from 0 to 100 with a visible breakdown
"Will an ATS read my resume?"	Five-pillar ATS audit
"Which roles fit me?"	Career path matching with match percentage, salary range and missing skills

Document status

Field	Value
Product	Futrix AI
Owner	Kirtan
Repository	github.com/kirtan597/Futrix-Ai
Release state	Deployed (Render and Netlify)
Document type	Technical product requirements document

2. Goals, non-goals and success metrics

The product succeeds when a user can go from an uploaded resume to a clear, trustworthy next step in a single session. The targets below are proposals for the owner to confirm; the README does not state numeric targets.

Goals

1. Give every user a repeatable, explainable analysis of their resume.
2. Show exactly which skills are missing for a target role and what to learn next.

3. Tell users whether an ATS is likely to parse their resume correctly.
4. Keep user data private through short-lived tokens, owner-only access and a private AI tier.
5. Stay cheap and simple to run: no per-request model cost, deployable on free-tier hosting.

Non-goals (current release)

- Parsing PDF or DOCX files in the browser (only `.txt` is read today).
- Generating or rewriting resume content with a language model.
- Recruiter or employer accounts, and live job-board integration.
- Guaranteeing a hiring outcome or a specific ATS vendor's behavior.

Success metrics

Metric	Definition	Proposed target
Reproducibility	Same resume text returns an identical result on repeat runs	100%
Skill precision	Reported skills that actually appear in the resume text	100% (closed vocabulary)
Completion rate	Users who upload a resume and reach the dashboard	To be set after first usage data
Analysis latency	Time from upload request to saved result, warm services	To be measured; set a budget once baseline is known
Return usage	Users who run a second analysis after improving their resume	To be set after first usage data
Availability	Share of `/health` checks that pass	To be set; free tier limits what is realistic

3. Target users and use cases

The primary user is a student or early-career job seeker aiming at a technical role. The README does not define personas, so the table below is a working definition for the owner to refine.

Users

User	Situation	What they need from Futrix AI
Student or fresher	Preparing first applications, unsure which skills employers expect	A gap list and learning roadmap for a target role
Career switcher	Moving into a technical role from another field	Role matches with the skills still missing
Working engineer	Updating a resume for a new job	An ATS audit and a readiness score to track improvement
Mentor or placement officer	Reviewing many resumes	Consistent, comparable results across candidates

Core use cases

ID	Use case	Outcome
UC-1	Sign in with email or Google	Authenticated session with a 15-minute access token
UC-2	Upload a plain-text resume	Saved analysis with skills, gaps, score and career paths
UC-3	Review skill gaps	List of missing skills and a roadmap
UC-4	Run an ATS check	Result on five pillars with what to fix
UC-5	Explore career paths	Match percentage, salary range and missing skills per role
UC-6	Compare two analyses	Visible change in score and skills after edits
UC-7	Review history	List of the user's last 20 analyses
UC-8	Manage profile	Updated name or avatar

4. Uniqueness

Futrix AI's edge is trust: it is the only approach in this comparison that is deterministic, never invents a skill and keeps resume text away from third-party AI models, while still covering skills, ATS and careers in one flow. The comparison is by approach, not by named competitor, and is a general characterization rather than a measured benchmark.

Approach comparison

Dimension	Futrix AI (rule-based, deterministic)	Generative-model resume tools	Keyword-only ATS checkers
Repeatability	Same resume gives the same result	Output can vary between runs	Usually repeatable
Invented skills	None; reports only skills found in text	Possible	None
Explainability	Every result traces to a rule or database entry	Hard to trace	Simple, but shallow
Per-analysis model cost	None	Charged per call	None
Resume text sent to a third-party model	No	Typically yes	Varies
Skill gaps and roadmap	Yes	Yes	Rarely
Career path match with salary range	Yes	Varies	No
ATS five-pillar audit	Yes	Varies	Yes, often keyword-focused
Understands synonyms and context	Limited to the database	Strong	Limited

What makes it distinct

1. **Zero-hallucination analysis.** Skills come from a closed database of 40+ technologies matched with word-boundary regular expressions, so an output can always be traced to text in the resume.
2. **One flow, five answers.** Skills, gaps, readiness score, ATS audit and career paths come from a single upload instead of five separate tools.
3. **Audit-friendly score.** The readiness score is a composite of four named components, `skill_match`, `stack_balance`, `cloud_presence` and `devops_score`, so users see why they scored as they did.
4. **Private AI tier.** The analysis service is never exposed to the browser; only the Node API can call it, using a shared secret.
5. **Hardened session model.** Short-lived access tokens, database-tracked refresh tokens with rotation, owner-only data access and per-route rate limits.
6. **Low operating cost.** No model calls per request, so it runs on free-tier hosting.

Honest trade-offs

The same design that removes hallucination also limits coverage. Skills outside the database are not detected and synonyms are not understood. The roadmap addresses this with a larger database and, optionally, semantic matching layered on top of the deterministic baseline.

5. Functional requirements

The release must satisfy 18 numbered requirements. Priority P0 means the product does not work without it, P1 means expected at launch, P2 means a later improvement. Acceptance criteria are written from the behavior stated in the README.

Authentication and profile

ID	Requirement	Priority	Acceptance criteria
FR-1	Users can sign in with email/password or Google through Firebase	P0	A valid Firebase ID token at `POST /api/auth/firebase` returns an access and refresh token and the user record

ID	Requirement	Priority	Acceptance criteria
FR-2	Access tokens expire after 15 minutes and refresh tokens after 7 days	P0	An expired access token is rejected; a valid refresh token yields a new pair
FR-3	Each refresh rotates the refresh token	P0	After a refresh, the previous refresh token is revoked and cannot be reused
FR-4	Users can sign out	P1	`POST /api/auth/logout` revokes the refresh token server-side
FR-5	Users can view and update their profile	P2	`GET /api/profile` returns the user; `PUT /api/profile` updates name or avatar

Analysis

ID	Requirement	Priority	Acceptance criteria
FR-6	The system accepts plain-text resumes of 50 to 50,000 characters	P0	Shorter or longer input is rejected with a client error
FR-7	The system extracts skills from the resume using a curated database	P0	Only skills present in `skills_db.json` and in the text are returned
FR-8	The system lists skill gaps against role benchmarks and a roadmap	P0	Response includes `gap_skills` and `roadmap`
FR-9	The system computes a 0–100 readiness score with a breakdown	P0	Response includes `readiness_score` and `score_breakdown` with `skill_match`, `stack_balance`, `cloud_presence`, `devops_score`
FR-10	The system matches career paths	P1	Each path has match percentage, salary range and missing skills

ID	Requirement	Priority	Acceptance criteria
FR-11	The system audits ATS readiness on five pillars	P1	Result covers formatting risk, section presence, date parseability, keyword match, file integrity
FR-12	The system returns chart data for the dashboard	P1	Response includes `skill_weights`, `category_distribution` and `readiness_trajectory`
FR-13	The same input produces the same output	P0	Repeat analyses of one resume return identical results

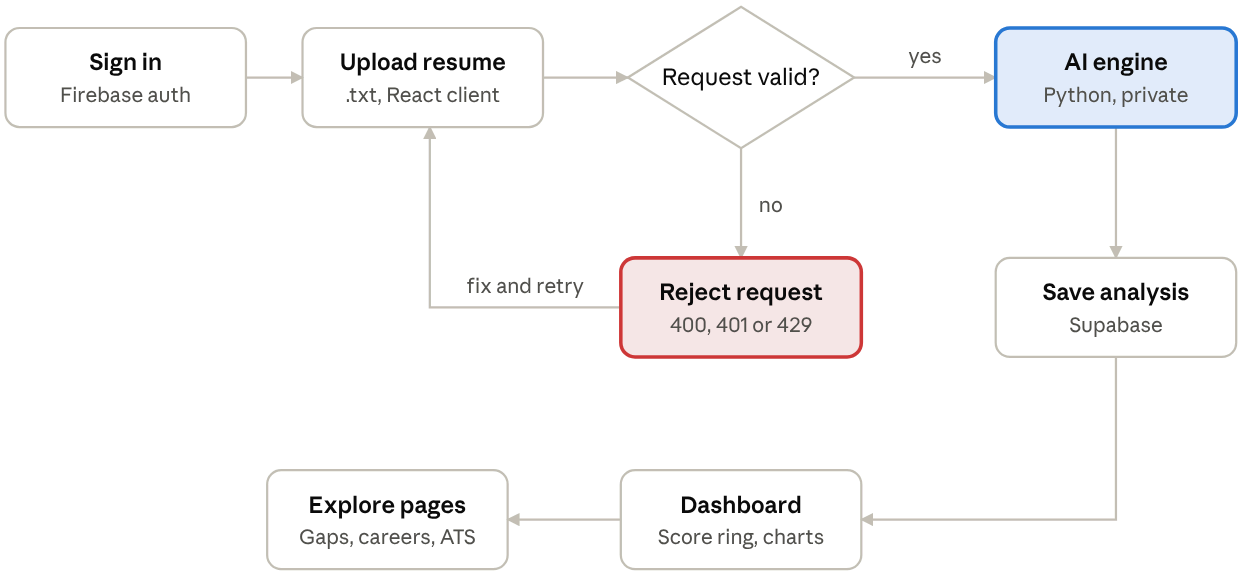
History and comparison

ID	Requirement	Priority	Acceptance criteria
FR-14	Every analysis is saved against the user	P0	The analysis is stored with the user's ID and returned by history
FR-15	Users can list their last 20 analyses	P1	`GET /api/history` returns at most 20 analyses, scoped to the JWT's user
FR-16	Users can compare two of their own analyses	P1	`GET /api/compare?id1=&id2=` succeeds only if both belong to the caller
FR-17	Users can match a skills list to jobs	P2	`POST /api/jobs/match` returns matches for a skills array
FR-18	Users cannot read another user's data	P0	Requests for another user's analysis ID are denied

6. User flow

Every analysis passes one gate before the AI engine runs: the Node API checks the token, rate limit and resume length, and a failed check returns an error the user can fix and retry.

A request either reaches the dashboard or is rejected before analysis



user flow · 7 steps, 1 decision

The highlighted engine step runs only after the request is valid. The saved result feeds the dashboard and every explore page, so later visits read history instead of re-running the analysis.

Flow steps and requirements

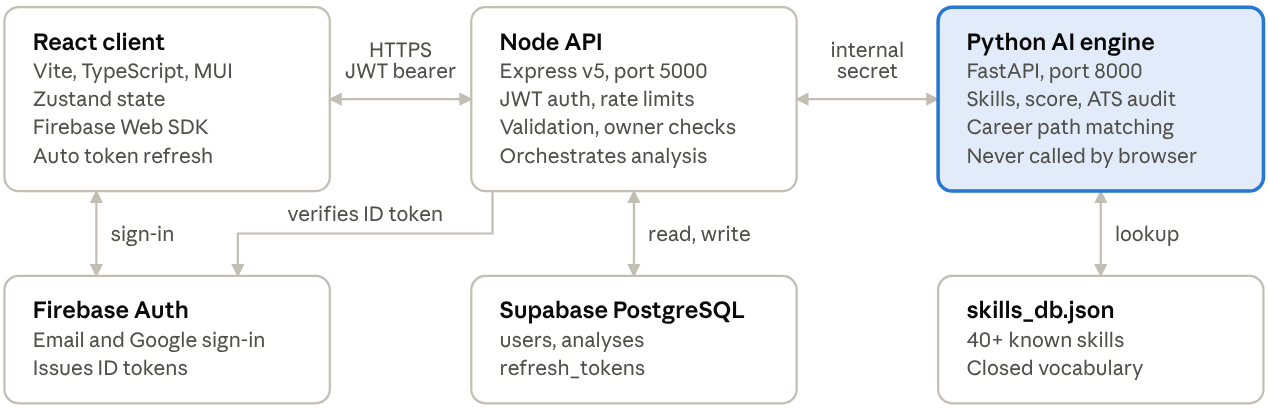
Step	What happens	Requirements
Sign in	Firebase verifies identity; the API issues a JWT pair	FR-1, FR-2
Upload resume	Client reads a `.txt` file and posts the text	FR-6
Request valid?	JWT check, rate limiter, length check	FR-2, FR-6, NFR-5, NFR-8
Reject request	400 for bad input, 401 for bad token, 429 for too many requests	NFR-5, NFR-8
AI engine	Extracts skills, gaps, score, ATS audit and career paths	FR-7 to FR-13
Save analysis	Stored under the user's ID	FR-14

Step	What happens	Requirements
Dashboard and explore pages	Render score, charts, gaps, careers, ATS; history and compare	FR-10 to FR-12, FR-15, FR-16

7. System architecture

Futrix AI is a three-tier system of a React client, a Node.js API and a Python AI engine, supported by Firebase Auth, Supabase PostgreSQL and a skills database. The Node API is the only public backend; it authenticates users, validates input and calls the Python engine on their behalf.

The browser reaches the AI engine only through the Node API



architecture · 3 services, 3 supporting systems

Read the diagram left to right: the browser calls the Node API with a JWT, and the Node API calls the Python engine with a shared secret. The highlighted engine has no direct route from the browser.

Component responsibilities

Component	Owns	Does not own
React client	Screens, state, charts, Firebase sign-in, token refresh	Business rules, scoring

Component	Owns	Does not own
Node API	Authentication, token issue and rotation, validation, rate limiting, owner checks, persistence, calling the AI engine	Language analysis
Python AI engine	Skill extraction, gap detection, scoring, ATS audit, career path matching	Users, sessions, storage
Supabase	Durable storage of users, analyses and refresh token hashes	Business logic
Firebase Auth	Identity: passwords, Google OAuth, ID tokens	Application sessions

Middleware order (Node API)

`cors` → `json` → `auth (JWT)` → `rateLimiter` → route → error handler.

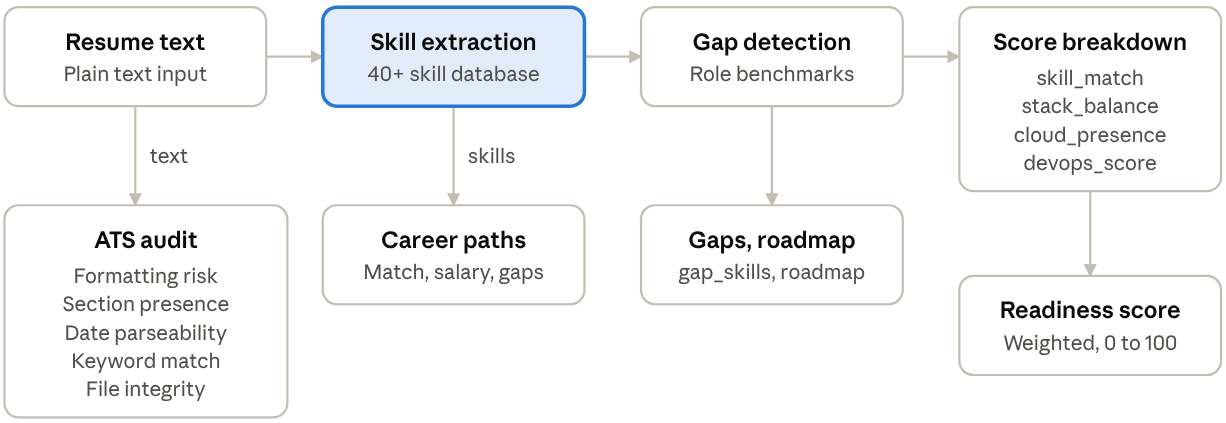
Design decisions

Decision	Reason
Node API in front of Python	One public entry point for auth, validation and rate limiting; the AI service stays private
Python for analysis	Natural language processing is simpler to build and extend in Python
Stateless services	Each service can be scaled or replaced independently
Managed identity (Firebase)	Password storage and Google OAuth are handled by a provider
In-memory database fallback	The stack runs locally without a Supabase project

8. AI engine specification

The engine is rule-based and deterministic: it matches resume text against a curated skill database and applies fixed scoring logic, with no generative model in the path. The code is in `python-ai/ai_engine.py` and the vocabulary in `python-ai/skills_db.json`.

Skill extraction against a closed database anchors the analysis



analysis pipeline · 8 steps, 2 branches

The highlighted extraction step is the anchor: gaps, the career path match and the readiness score all use the skills it finds, while the ATS audit runs on the resume text itself.

Processing rules

Stage	Rule	Result
Skill extraction	Word-boundary regular expressions plus substring match against `skills_db.json` (40+ technologies)	Detected skills; nothing outside the database is reported
Gap detection	Detected skills compared with benchmark skills for each role	`gap_skills` and `roadmap`
Score breakdown	Four components: `skill_match`, `stack_balance`, `cloud_presence`, `devops_score`	`score_breakdown`
Readiness score	Weighted composite of the components	0–100 `readiness_score`
Career path match	Role catalog compared with detected skills	Match percentage, salary range, missing skills per role
ATS audit	Five heuristic pillars (below)	Pass or fail detail per pillar

ATS five-pillar audit

Pillar	Checks
Formatting risk	Layout patterns that commonly break applicant tracking parsers
Section presence	Whether expected resume sections exist
Date parseability	Whether dates are in a consistent, readable format
Keyword match	Overlap between resume terms and role keywords
File integrity	Whether the submitted content is readable and complete

Engine guarantees

1. **Deterministic:** identical input always yields identical output (FR-13).
2. **Closed vocabulary:** only skills in the database can be reported (FR-7).
3. **Authenticated:** every endpoint except `/health` checks `X-Internal-Secret`.
4. **Bounded input:** the Node API passes at most 50,000 characters.

The exact weights of the readiness score are not stated in the README and are tracked as an open question in section 14.

9. Technical specification

The system exposes 12 Node API routes plus `/health`, six internal Python endpoints and three database tables. Request and response bodies are not specified in the README, so this section documents routes, rules and storage; schemas should be added from the source code.

Node API contract

Base URL: `https://futrix-node-api.onrender.com`. Middleware order: CORS, JSON parsing, JWT auth, rate limiter, route, error handler.

Method	Endpoint	Auth	Purpose
POST	`/api/login`	No	Email login, returns a JWT pair

Method	Endpoint	Auth	Purpose
POST	`/api/auth/firebase`	No	Firebase ID token to JWT pair
POST	`/api/auth/refresh`	No	Rotate refresh token; old one invalidated
POST	`/api/auth/logout`	No	Revoke refresh token
GET	`/api/auth/verify`	Yes	Validate JWT, return user profile
GET	`/api/profile`	Yes	Fetch profile
PUT	`/api/profile`	Yes	Update name or avatar
POST	`/api/upload-resume`	Yes	Analyze, save and return AI data
GET	`/api/history`	Yes	Last 20 analyses for the caller
GET	`/api/compare?id1=&id2=`	Yes	Compare two analyses, owner-checked
POST	`/api/jobs/match`	Yes	Match a skills array to the job database
POST	`/api/ats-check`	Yes	ATS audit through the Python engine
GET	`/health`	No	Service health and database status

Python AI engine contract (internal)

All endpoints except `/health` require header `X-Internal-Secret` equal to `INTERNAL_API_SECRET`.

Method	Endpoint	Purpose
POST	`/analyze`	Full resume NLP analysis
POST	`/ats-check`	Five-pillar ATS audit
POST	`/score-breakdown`	Score components only
POST	`/career-path`	Career paths from a skills list
POST	`/compare`	Compare two resume texts
GET	`/health`	Health check

Rate limits

Endpoint	Limit	Window
`POST /api/login`	10 requests	15 minutes
`POST /api/auth/firebase`	20 requests	15 minutes
`POST /api/upload-resume`	100 requests	15 minutes

Data model

Table	Key columns	Notes
`users`	`id` (UUID), `email` (unique), `name`, `auth_provider`, `firebase_uid`, `google_id`, `avatar`, `login_attempts`, `lock_until`, `last_login`, timestamps	One row per person
`analyses`	`id`, `user_id`, `email`, `resume_text`, `skills[]`, `gap_skills[]`, `readiness_score`, `roadmap[]`, `score_breakdown`, `career_paths`, `skill_weights`, `category_distribution`, `readiness_trajectory`, timestamps	One row per analysis; arrays are TEXT[], structured results are JSONB
`refresh_tokens`	`id`, `user_id`, `token_hash`, `expires_at`, `revoked_at`, `created_at`	Only a hash is stored

Without Supabase credentials the Node API falls back to an in-memory store for development; data is lost on restart.

Token lifecycle

Token	Algorithm	Lifetime	Storage
Access JWT	HS256	15 minutes	Client `localStorage` and Zustand

Token	Algorithm	Lifetime	Storage
Refresh JWT	HS256	7 days	Client storage; hash in `refresh_tokens`

On expiry the client's `apiService.ts` calls `/api/auth/refresh`; the old refresh token is revoked and a new pair stored.

10. Non-functional requirements

Security is the strongest area of the current design; performance, reliability and compliance targets are not stated in the README and are proposed here for the owner to confirm.

Security

ID	Requirement	Mechanism
NFR-1	Identity is handled by a managed provider	Firebase Auth, email/password and Google OAuth
NFR-2	Stolen access tokens have a short useful life	JWT HS256, 15-minute lifetime
NFR-3	Refresh tokens are revocable and single-use	7-day lifetime, database-tracked, rotated on every refresh
NFR-4	Users cannot read each other's data	Owner check on all user-scoped routes (IDOR protection)
NFR-5	Abuse is throttled	Sliding-window rate limits: 10 login, 20 Firebase auth, 100 upload per 15 minutes
NFR-6	The AI engine is not publicly callable	`X-Internal-Secret` on every endpoint except `/health`
NFR-7	Only trusted origins can call the API	Strict CORS whitelist, no wildcard
NFR-8	Malformed or oversized input is rejected	Resume text limited to 50 to 50,000 characters
NFR-9	Secrets never enter source control	Environment variables only; `.env` and `serviceAccountKey.json` excluded

Performance, reliability and operations (proposed)

ID	Requirement	Note
NFR-10	Analysis is cheap enough to run synchronously	Regex matching over at most 50,000 characters; no external model call
NFR-11	Cold starts are mitigated	`serviceWarmer.js` keeps the Python service warm in production
NFR-12	Each service reports health	`GET /health` on Node (with database status) and Python

ID	Requirement	Note
NFR-13	Services are stateless and scale independently	State lives in Supabase; note the rate limiter is per process
NFR-14	Local development works without cloud setup	In-memory database fallback
NFR-15	Results are reproducible	No randomness or model sampling in the analysis path

Privacy and compliance (to be decided)

Resume text is personal data and is stored in `analyses.resume_text`. The README defines no retention period, deletion path or privacy policy. Before wider launch, define how long resumes are kept, how a user deletes their data, and which regional privacy rules apply to the user base.

11. Deployment, environments and configuration

Production uses Render for the two backend services and Netlify for the client, configured by `render.yaml`. Local development runs all three services on one machine with the Vite proxy forwarding `/api/*` to port 5000.

Environments

Environment	Client	Node API	Python AI	Database
Local	`localhost:5173` (Vite)	`localhost:5000`	`localhost:8000`	Supabase or in-memory fallback
Production	Netlify, `futrixai.netlify.app`	Render, `futrix-node-api.onrender.com`	Render, `futrix-python-ai.onrender.com`	Supabase PostgreSQL

Services

Service	Runtime	Root directory	Start command
`futrix-node-api`	Node 18	`node-api/`	`node server.js`
`futrix-python-ai`	Python 3.11	`python-ai/`	`uvicorn main:app --host 0.0.0.0 --port $PORT`

Configuration reference

Service	Variable	Purpose
Node API	`NODE_ENV`	`production` in deployed environments
Node API	`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Database access
Node API	`JWT_SECRET`, `JWT_REFRESH_SECRET`	Token signing
Node API	`INTERNAL_API_SECRET`	Shared secret for calls to Python
Node API	`PYTHON_URL`	Base URL of the Python engine
Node API	`FRONTEND_URL`	Allowed browser origin
Node API	`FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL`, `FIREBASE_PRIVATE_KEY`	Firebase Admin SDK
Python AI	`INTERNAL_API_SECRET`	Must equal the Node API value
Python AI	`ALLOWED_ORIGINS`	Allowed callers
Client	`VITE_API_URL`	Node API base URL

Service	Variable	Purpose
Client	`VITE_FIREBASE_API_KEY`, `_AUTH_DOMAIN`, `_PROJECT_ID`, `_STORAGE_BUCKET`, `_MESSAGING_SENDER_ID`, `_APP_ID`	Firebase web config

Release checklist

- [ ] `INTERNAL_API_SECRET` is identical on both Render services
- [ ] `FRONTEND_URL` matches the Netlify domain exactly
- [ ] `PYTHON_URL` points to the deployed Python service
- [ ] Both `/health` endpoints return healthy
- [ ] A test login and resume upload work end to end
- [ ] No secrets or `.env` files are committed

12. Risks, assumptions and dependencies

The largest product risks are limited skill coverage and a `.txt`-only input format; the largest technical risks are token storage in the browser and a per-process rate limiter.

Risks

Risk	Impact	Likelihood	Mitigation
Skill database misses technologies	Gaps overstated, lower user trust	High	Grow the database; add aliases; show which skills were checked
Users have PDF or DOCX resumes only	Drop-off at upload	High	Add PDF and DOCX parsing
Tokens in `localStorage` readable by injected scripts	Session theft	Medium	Strict Content Security Policy, output sanitizing; consider HttpOnly cookie for refresh token

Risk	Impact	Likelihood	Mitigation
In-memory rate limiter resets and is not shared	Weaker abuse protection with multiple instances	Medium	Move to a shared store before scaling out
Free-tier cold starts	Slow first request, poor first impression	High	Service warmer today; paid tier or scheduled warm-up later
Email login route accepts email without proof of ownership (if implemented as described)	Account takeover	Unknown	Review `POST /api/login`; require verification or retire it
Resume text retained indefinitely	Privacy exposure	Medium	Define retention and a user deletion path
Single maintainer	Slow delivery, key-person dependency	Medium	Document, add tests and CI, enable reviews

Assumptions

- Users upload text they are allowed to share, and the text is complete enough to analyze (50 characters minimum).
- The skill database and role benchmarks are accurate enough for the target roles.
- Render and Netlify free tiers are sufficient for the expected early traffic.
- Firebase remains the identity provider.

Dependencies

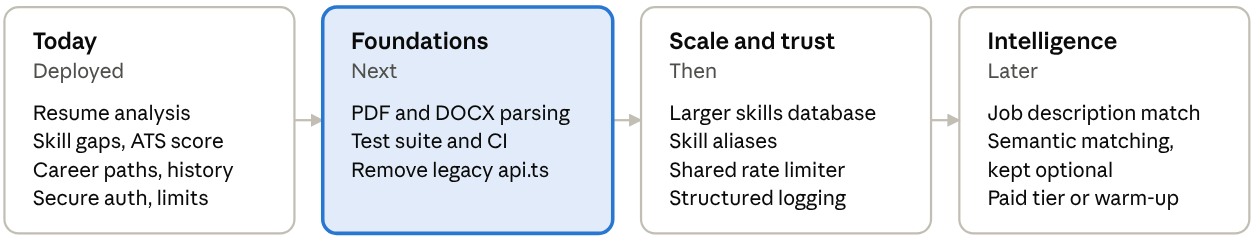
Dependency	Used for	Impact if unavailable
Firebase Auth and Admin SDK	Sign-in and ID token verification	Users cannot sign in
Supabase PostgreSQL	Users, analyses, refresh tokens	Falls back to memory in development; production loses persistence

Dependency	Used for	Impact if unavailable
Render	Hosting Node API and Python engine	Analysis and API unavailable
Netlify	Hosting the client	Application interface unavailable
`skills_db.json`	Skill vocabulary	No skills detected

13. Release plan and roadmap

The current release is deployed; the phases after it are a proposal for the owner to confirm and date, and the README states no dates or commitments.

The next phase adds real-world resume formats and safe releases



Proposed order, not dated

roadmap · 4 phases, proposed order

The highlighted phase is the next one: it removes the two biggest blockers to adoption and trust, the plain-text-only input and the lack of automated tests.

Phase scope and exit criteria (proposed)

Phase	Scope	Exit criteria
Today	Resume analysis, skill gaps, ATS audit, career paths, history and compare, Firebase plus JWT auth with rotation, rate limits	Deployed on Render and Netlify
Foundations	PDF and DOCX parsing; automated unit, API and end-to-end tests with CI; retire the legacy `api.ts` axios client	A PDF resume analyzes end to end; CI blocks a failing change

Phase	Scope	Exit criteria
Scale and trust	Larger skills database with aliases; shared rate-limit store; structured logging with request IDs; retention and deletion for resume text	Two Node instances share limits; every request is traceable across Node and Python
Intelligence	Match a resume to a job description; optional semantic matching layered on the deterministic baseline; paid hosting tier or scheduled warm-up	Deterministic results stay reproducible with semantic matching turned off

Release rules

1. Do not ship a change to authentication, tokens or the internal secret without a second reviewer.
2. Run both `/health` checks and one full upload before every production release.
3. Update this document's requirements and API tables in the same change as the code.

14. Open questions and appendix

Open questions

- [ ] What are the exact weights of the four readiness score components, and where are role benchmarks defined?
- [ ] Should `POST /api/login` (email login) stay, and what proves email ownership?
- [ ] What is the retention period for `resume_text`, and how does a user delete their data?
- [ ] Which numeric targets apply to completion rate, latency and availability?
- [ ] Which target roles must the first skill database expansion cover?
- [ ] Is an HttpOnly cookie for the refresh token acceptable for the deployed domains?

Technology stack

Layer	Technology
Frontend	React 18, TypeScript, Vite, Material UI v5

Layer	Technology
State and charts	Zustand, Recharts, Framer Motion
Client auth	Firebase Web SDK v10
HTTP client	`apiService.ts` (fetch with automatic token refresh)
API server	Node.js 18, Express v5
Server auth	Firebase Admin SDK, jsonwebtoken
Database	Supabase PostgreSQL, in-memory fallback for development
AI engine	Python 3.11, FastAPI, Uvicorn
Deployment	Render (Node and Python), Netlify (React)

Client pages

Route	Page	Data source
`/login`	Login	`POST /api/auth/firebase`
`/upload`	Upload resume	`POST /api/upload-resume`
`/dashboard`	Dashboard	`localStorage` (`analysisResult`)
`/result`	Resume result	`localStorage` , `GET /api/history`
`/skills-gap`	Skills gap	`localStorage` , `GET /api/history`
`/career-path`	Career path	`localStorage` , `GET /api/history`
`/ats-check`	ATS audit	`POST /api/ats-check`
`/history`	History	`GET /api/history`
`/profile`	Profile	`GET /api/profile` , `GET /api/history`

Repository map

Path	Contents
`client/`	React app: pages, services, Zustand stores, components, `vite.config.ts`
`node-api/`	Express server, `routes/userRoutes.js`, auth and rate-limit middleware, Supabase repositories, utilities
`python-ai/`	`main.py`, `ai_engine.py`, `skills_db.json`
`render.yaml`	Render deployment configuration
`.agents/rules/PROJECT_RULES.md`	Mandatory project coding rules

Contribution rules

1. All client API calls must use `apiService` from `services/apiService.ts`.
2. All protected Node routes must use the `auth` middleware.
3. All Python endpoints except `/health` must validate `X-Internal-Secret`.
4. Never commit `.env` files or `serviceAccountKey.json`.